ENGENHARIA DE SOFTWARE / DEVSECOPS AND CLOUD COMPUTING

O Pipeline DevSecOps Completo

Gitleaks · Semgrep · Trivy

Checkpoint 05 | 13 de outubro de 2026

Aluno: Enzo Sartorelli | RM: _____

01 / VISÃO GERAL

Introdução

Este checkpoint integra verificações automáticas de segurança ao fluxo de integração contínua. Cada push e pull request aciona uma análise de segredos, uma análise estática de JavaScript e uma análise de vulnerabilidades da imagem Docker. Detectar problemas cedo reduz o risco de publicar credenciais, executar código não confiável e distribuir componentes vulneráveis.

Arquitetura do pipeline

O arquivo **.github/workflows/security.yml** define três jobs independentes. O Gitleaks lê o histórico da branch após checkout completo; o Semgrep inspeciona o código-fonte com uma regra local; o Trivy constrói a imagem do Dockerfile e consulta bases de CVEs. Jobs independentes permitem ver qual barreira falhou sem confundir o tipo de problema.

push / pull request → gitleaks + semgrep + trivy

Política: Gitleaks bloqueia segredos; Semgrep bloqueia eval; Trivy exibe todas as severidades e bloqueia CRITICAL.

Aplicação analisada

O repositório começou vazio. Foi criada uma aplicação HTML/CSS/JavaScript sem dependências de produção, servida por Nginx sem privilégios na porta interna 8080. O painel apresenta as três barreiras e uma lista local de estudo salva no navegador. O progresso da interface não representa o estado dos jobs.

Estado de validação

A aplicação respondeu por HTTP local e o JavaScript passou na verificação de sintaxe. O workflow YAML foi interpretado com os gatilhos e três jobs esperados. O ambiente não dispõe de Docker nem de remoto GitHub configurado; por isso o build, a execução do workflow no GitHub e as capturas exigidas seguem pendentes.

02 / PRIMEIRA BARREIRA

Gitleaks: segredos no histórico

O Gitleaks procura credenciais expostas em commits. O job instala a versão 8.30.0, usa **fetch-depth: 0**, estende as regras padrão e acrescenta uma regra local que identifica apenas o formato fictício do teste. A execução usa saída 1 em caso de achado e redige o valor do segredo no log.

Sabotagem isolada

A branch **prova/gitleaks** recebeu um commit com um token inteiramente fictício no arquivo de teste. A análise local percorreu dois commits, encontrou um achado com **RuleID fiap-cp05-fake-token** e terminou com **código 1**. A branch main não contém o arquivo de sabotagem. O log autêntico está em **evidencias/gitleaks-local.txt**.

Resultado local confirmado

1 achado · RuleID fiap-cp05-fake-token · saída 1 · commit c68a4d5. Trata-se de falha esperada de segurança, não de erro de configuração.

Captura do GitHub Actions: PENDENTE

Não há execução remota confirmada. A captura 01-gitleaks.png deve ser obtida do job gitleaks após publicar a branch isolada. Nenhum print foi fabricado.

Por que importa

Segredos em commits podem continuar acessíveis mesmo após a remoção do arquivo em uma revisão posterior. A resposta a um segredo real seria revogá-lo ou rotacioná-lo e avaliar o histórico afetado. Este teste usa exclusivamente um valor fictício.

03 / SEGUNDA BARREIRA

Semgrep: análise estática

O Semgrep 1.140.0 examina os arquivos JavaScript do repositório. A regra local **fiap-cp05-no-eval**, criada especificamente para a demonstração, identifica chamadas **eval(...)**. A opção **--error** faz um achado resultar em falha do job.

Exemplo inserido somente na branch prova/semgrep

```
function exemploInseguro(entrada) { return eval(entrada); }
```

A função acima não é chamada na demonstração. A análise local apontou **1 finding** na linha 52 de app.js com a regra **semgrep.fiap-cp05-no-eval** e terminou com **código 1**. Na branch main, a mesma varredura terminou com zero achados e código 0. O log autêntico está em **evidencias/semgrep-local.txt**.

Resultado local confirmado

1 achado bloqueante na branch de prova · saída 1 · commit 0225a55. O código vulnerável não foi incorporado à main.

Captura do GitHub Actions: PENDENTE

A captura 02-semgrep.png precisa mostrar a falha real do job semgrep depois que a branch de prova for publicada.

Risco de eval

eval interpreta texto como código JavaScript. Quando a entrada pode ser influenciada por terceiros, a execução pode alterar o comportamento esperado da aplicação e permitir acesso indevido a dados ou funções do contexto.

04 / TERCEIRA BARREIRA

Trivy: vulnerabilidades da imagem

O job trivy constrói **fiap-cp05:${{ github.sha }}** a partir do Dockerfile do repositório. Depois gera um JSON como artefato, mostra uma tabela com todas as severidades nos logs e executa a política final que retorna código 1 somente se houver vulnerabilidades **CRITICAL**. Nenhuma vulnerabilidade é ignorada para limpar artificialmente o relatório.

Observação local da imagem-base

Como não havia Docker disponível, o Trivy 0.75.0 analisou diretamente a imagem-base remota **nginxinc/nginx-unprivileged:stable-alpine** em 13/10/2026. O digest observado foi **sha256:56b8272e43a0525f67aed68d1e0794a712d10e3e45a563e118e9a77c817e6f86**. Esse teste **não valida a imagem do projeto**, que ainda precisa ser construída.

Achado real da imagem-base

CVE-2026-85091 · MEDIUM · zlib 1.3.2-r0 · correção indicada: 1.3.2-r1. Nenhuma CRITICAL foi encontrada nessa imagem-base; a verificação local com --severity CRITICAL --exit-code 1 retornou 0.

Interpretação

CVE é um identificador público de vulnerabilidade. A severidade MEDIUM indica risco abaixo do limite de bloqueio definido para o CP05, mas ainda deve ser acompanhada e corrigida quando uma atualização segura estiver disponível. A tag da imagem-base pode mudar; uma execução posterior pode apresentar outros resultados. O log local consta em **evidencias/trivy-base-local.txt**.

Captura e resultado da imagem construída: PENDENTES

A captura 03-trivy.png, os CVEs da imagem do projeto e a confirmação do status do job dependem de build Docker e execução do GitHub Actions.

05 / ENCERRAMENTO

Conclusão

As três ferramentas protegem etapas diferentes: Gitleaks examina exposição de credenciais no histórico; Semgrep identifica um padrão de código inseguro; Trivy relaciona pacotes da imagem a CVEs. As sabotagens locais confirmaram o bloqueio do Gitleaks e do Semgrep. A varredura local do Trivy confirmou seu funcionamento contra a imagem-base, com um achado MEDIUM; a validação da imagem construída permanece pendente.

Repositório e execuções

URL do GitHub: PENDENTE

O diretório local ainda não possui remoto. Não há links de runs para registrar. Após a publicação, incluir aqui a URL do repositório e as URLs das três execuções usadas como evidência.

Pendências para a entrega integral

1. Construir e executar a imagem Docker do projeto.
2. Publicar main e as duas branches de prova sem mesclá-las.
3. Confirmar os três jobs no GitHub Actions, incluindo CVEs e política do Trivy.
4. Capturar 01-gitleaks.png, 02-semgrep.png e 03-trivy.png a partir de runs reais.
5. Acrescentar URLs, resultados remotos e imagens autênticas a este relatório.

Fontes técnicas

Gitleaks: github.com/gitleaks/gitleaks
Semgrep CLI: semgrep.dev/docs/cli-reference
Trivy Action: github.com/aquasecurity/trivy-action
Nginx sem privilégios: github.com/nginx/docker-nginx-unprivileged